

Post Graduate Diploma in Information Technology (PGDIT)

Institute of IT, University of Dhaka, Bangladesh

Course Name: Data Structure # Course Code: PGD 216

Linked Lists

Linked lists represent a fundamental shift in how we think about storing data. Unlike arrays, which behave like a row of lockers in a hallway, a Linked List is more like a scavenger hunt each item tells you where the next one is hidden.

A Singly Linked List is a dynamic linear data structure. While an array reserves a contiguous block of memory and uses indices to access elements, a Linked List is made up of individual objects called **nodes** that are scattered throughout memory. Each node is logically connected to the next through a **reference** (or pointer). This design allows efficient memory usage because the list grows as needed without requiring a large initial block of memory.

Example:

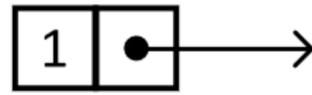
Imagine you have a case of cola that you want to store in your refrigerator. Your first idea might be to place the entire box inside the fridge. This makes it easy to find the drinks and keep track of how many are left because they're all in one place. However, if your refrigerator is already filled with groceries, you might not have a space large enough to fit the whole case.

But if you open the case and store each can individually, you can likely find small spots to fit all 12 cans. Now all the cans are cold, but locating them becomes more difficult than when they were stored together in the box.

Structure of Linked Lists

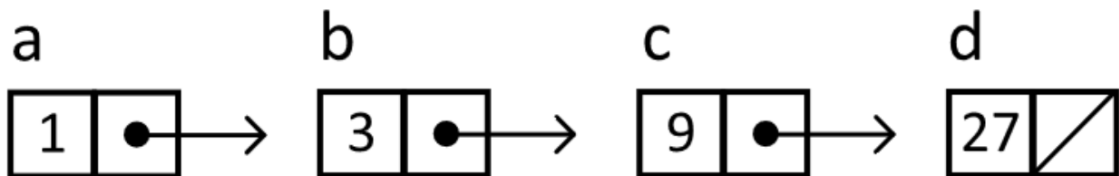
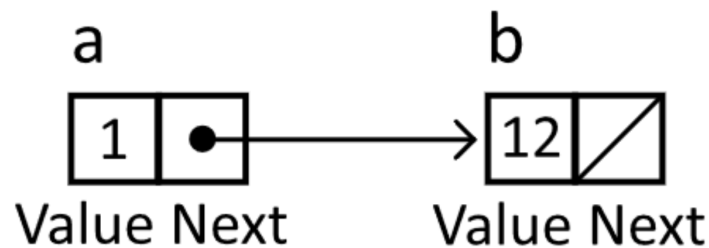
Linked Lists do not use contiguous memory, which means they require a different way to maintain the sequence of elements. Instead of relying on index-based positions like arrays, Linked Lists (and later, trees and graphs) use the concept of a node. Each node contains a reference to another node, forming a chain-like structure. By combining these references with a stored value (often called a payload), we create a data structure that is fundamentally different from arrays and comes with its own unique advantages and trade-offs.

Node



Value Next

```
1 class Node
2     Integer Value
3     Node Next
```



```
1 set a to a new instance of Node
2 set a.Value to 1
3 set b to a new instance of Node
4 set b.Value to 3
5 set c to a new instance of Node
6 set c.Value to 9
7 set d to a new instance of Node
8 set d.Value to 27
9 set a.Next = b
10 set b.Next = c
11 set c.Next = d
```

Comparing with Array and Linked List

Feature	Array	Linked List
Size	Fixed (Static)	Dynamic (Grows at runtime)
Access	Fast ($O(1)$ via index)	Slow ($O(n)$ - must traverse)
Insertion/Deletion	Slow (requires shifting)	Fast (just change pointers)
Memory	Contiguous block	Scattered (Non-contiguous)

Defining the Node

Every node is an object. It needs a place to store data and a variable to hold the reference to the next object.

```
class Node:
    def __init__(self, data):
        self.data = data # The value
        self.next = None # The pointer to the next node
```

The Linked List Class

The "List" itself just needs to keep track of where the first node (the **Head**) is.

```
class LinkedList:
    def __init__(self):
        self.head = None # List starts empty
```

Insertion

This is the most efficient insertion (). You create a new node and make it the new head.

```
# 1. Insertion at the Beginning (O(1))

def insert_at_head(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node

# 2. Insertion at the End (O(n))

def insert_at_end(self, data):
    new_node = Node(data)
    if not self.head:
        self.head = new_node
        return

    last = self.head
    while last.next: # Traverse to the end
        last = last.next
    last.next = new_node
```

Deletion

To delete a node by value, you have to look for the node **before** the one you want to remove.

```
def delete_by_value(self, key):  
    current = self.head  
  
    # Case 1: Deleting the Head  
    if current and current.data == key:  
        self.head = current.next  
        current = None  
        return  
  
    # Case 2: Searching for the key  
    prev = None  
    while current and current.data != key:  
        prev = current  
        current = current.next  
  
    # If key not found  
    if current is None:  
        print("Value not found in list.")  
        return  
  
    # Unlink the node  
    prev.next = current.next  
    current = None
```